



PROJECT REPORT

Edu-Flow

Name: Itiksha Bhardwaj

Registration Number: 25BAI10069

Branch: B. Tech Computer Science Engineering (AI-ML)

Year: 2026-27

Course Title: Programming in Java

Course Code: CSE2006

Slot: F11+F12

Faculty Name: Dr. Manorma Chauhan

Class No.: BL2025260500820

1. Introduction

EduFlow is a Java application made to track of students attendance and exam in the education. Rather than doing perform tasks one by one, the application is geared towards having everything in one, such as attendance reports, student records, attendance tracking, etc.

The application uses Java like object-oriented programming (OOP), exceptions, file I/O, command-line interface, collection frameworks and input validation.

2. Problem statement

Doing all the processes related to student and student exam might take a lot of time and might cause calculations wrong of those processes. Attendance calculated wrongly; some records not included; mistakes in bookings of the exam rooms can occur.

EduFlow simplifies those by a simple app that automatically calculates the attendance percentage of students and checks if they meet the required attendance percentage to take exams.

3. Objectives

The main objectives of EduFlow are to:

- Manage student records.
- Calculate and update student attendance percentages on its own.
- Check if students meet the 75% attendance of exams.
- give a what-if calculator to see how many more classes a student needs to attend to improve their attendance.
- Generate attendance reports and identify students whose attendance is below 75%.
- Manage exam schedules and detect conflicts when the same room is assigned to two exams at the same time.
- Demonstrate the use of Java concepts such as OOP, collections, file handling, exception handling, and input validation.

4. Functional Requirements

1. Student Management

The system should allow users to:

- Add a student's name and department.
- Automatically generate a unique student ID.
- View all registered student records.
- Search for a particular student using their ID.

2. Attendance and Exam Management

The features of the system include:

- The ability to update attendance by entering the number of classes held and attended.
- The system must automatically compute attendance percentage.
- The system must allow checking if the student is eligible for exams based on the condition of 75% attendance.
- The system must determine how many additional classes the student must attend to achieve 75% attendance.
- The ability to schedule exams by inputting subject, date, time, and venue.
- The ability to validate the entered date and time, and make sure that there are no overlapping exams in the same venue at the same time.

3. Reporting

The system should be able to:

- Display the total number of students.
- Calculate the average attendance of all students.
- Count the number of students who are eligible for exams.
- Identify the student with the highest attendance percentage.
- List all students whose attendance is below 75%.

5. Non-Functional Requirements

The system is classified by:

- Usability: The program has a simple, menu-driven console that makes the navigation easy for its users.

- **Reliability:** Input verification and exception handling minimize the risk of the potential breakdown of the system when an inappropriate input is entered.
- **Maintainability:** The tasks are distributed among classes like `Student`, `Exam`, `Attendance`, `FileManager`, and `Eduflow`, which makes it easier to read the code and make changes in it.
- **Performance:** Due to the fact that the system deals with a small-sized dataset of students, `ArrayList` and the local text file storage are enough for the system.
- **Error Management:** Invalid numeric values and dates, times, and scheduling are dealt with without stopping the program immediately.
- **Resource Effectiveness:** The system uses only local file storage and standard Java libraries for the application's availability.

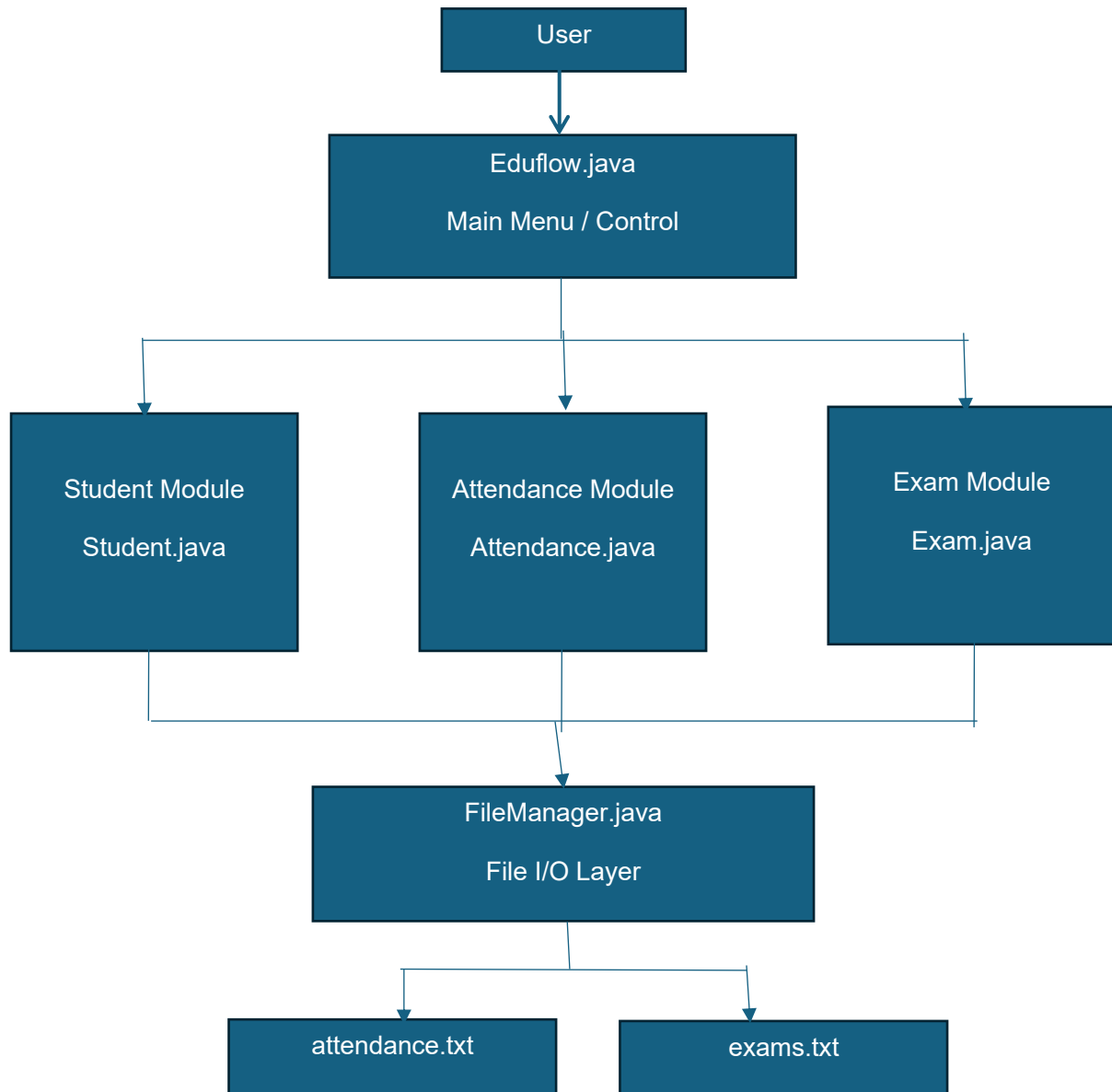
6. System Architecture

EduFlow uses a **simple modular architecture** in which different different classes do separate specific academic management tasks. The application uses a command-line interface for talking with user and stores information it gets on your computer where you use it in text files.

Component Description

Component	Responsibility / Function
Eduflow.java	Gives the main menu options and controls for the full program flow.
Student.java	Show student info and manages attendance-related calculations like percentage and eligibility.
Attendance.java	Take care of attendance updates, viewing attendance and what-if option in app.
Exam.java	Manages examination details and checks for scheduling conflicts.
FileManager.java	Saves and loads student and examination data using text files.
Text Files / Output	Provide simple local storage for attendance and examination records.

Architecture Diagram



Architecture Working

1. The **user** talks to with EduFlow with the menu we give.
2. Eduflow.java gets in the user's choice they give and calls the required module.
3. The **Student, Attendance and Exam modules** do their operations when called.
4. FileManager.java does saving and loading records- basically full manager of app.
5. The info is saved in computer of person in attendance.txt and exams.txt

8. Design Decisions & Rationale

1. Modular Class-Based Structure

Decision: The project is made in different classes called Eduflow, Student, Attendance, Exam, and FileManager.

Rationale: Giving different responsibilities makes the app easier to understand, maintain, use, and modify. It also shows and uses object-oriented programming concepts in Java.

2. Command-Line Interface

Decision: My app uses a menu which take input from you to work.

Rationale: I keeps the project lightweight and give the core Java functionality to be working without using more GUI frameworks or frontend work.

3. ArrayList for Data Management

Decision: ArrayList is used by me to store student and exam records.

Rationale: ArrayList gives big storage, allowing records to be added without saying a big fix size from before. It also show use of Java Collections.

4. File-Based Storage

Decision: Student and exams data are saved in text files using FileManager.

Rationale: File storage gives data without needing a database. It is suitable for the current use of my small project and show Java File I/O concepts.

5. Separate Attendance and Examination Modules

Decision: Attendance and examination operations are take care for separately.

Rationale: This follows keeping related use together. It makes uses such as attendance calculation, eligibility check, what-if find, and exam check easier for manage.

6. 75% Attendance Threshold

Decision: A 75% attendance threshold is used for eligibility.

Rationale: The border gives a clear rule for determining if a student meets the needed attendance level and show the system to perform meaningful attendance analysis and what-if calculations.

9. Implementation Details

- **Language:** Java
- **Interface:** Menu-driven Command Line Interface (CLI)
- **OOP:** Classes and objects are used to organize student, attendance, and exam data.
- **Collections:** ArrayList stores student and examination records.
- **File I/O:** FileWriter, FileReader, and BufferedReader provide data storage and retrieval.
- **Exception Handling:** try-catch blocks handle invalid input and file-related errors.
- **Validation:** Checks are performed to prevent invalid attendance values and examination conflicts.
- **Core Features:** Student management, attendance calculation, what-if analysis, and exam scheduling.

10. Screenshots / Results

1. Main Menu

```
===== EDUFLOW =====
1. Add Student
2. View Students
3. Update Attendance
4. View Attendance
5. Attendance What-If
6. Schedule Exam
7. View Exams
8. Save Data
9. Exit
=====
Enter choice: |
```

2. Add Student

```
=====
Enter choice: 1
Enter name: Itiksha
Enter department: AIML
Student added. ID: 3
```

3. Update Attendance

```
=====
Enter choice: 3
Enter student ID: 3
Total classes: 22
Classes attended: 20
Attendance: 90.91%
Eligible.
```

4. View Attendance

```
=====
Enter choice: 4
ID: 1 | Jess | 85.19% | Eligible
ID: 2 | John | 62.50% | Not Eligible
ID: 3 | Itiksha | 90.91% | Eligible
```

5. Attendance What-If

```
=====
Enter choice: 5
Enter student ID: 3
Already above 75%.
```

6. Schedule Exam

```
=====
Enter choice: 6
Subject: Java
Date (DD/MM/YYYY): 25/09/2026
Time (HH:MM): 15:32
Room: 534
Exam added successfully.
```

7. Save Data

```
=====
Enter choice: 8
Data saved successfully.
```

8. Exit

```
=====
Enter choice: 9
Data saved successfully.
Thank you for using EduFlow!
```

11. Testing Approach

I tested EduFlow using **functional testing** to verify that each major feature works correctly.

- **Student Management:** Tested how to add and view student records.
- **Attendance:** I checked the method for calculating attendance and verified the 75% eligibility criterion.
- **What-If Analysis:** A calculation has been carried out to determine the number of additional classes needed to achieve 75%.
- **Exam Scheduling:** I added the facility to schedule exams and was able to detect room and time conflicts.
- **File Handling:** Tested the handling of files by saving and loading the stored records.
- **Input Validation:** We tested invalid and empty inputs to make sure that suitable error messages were displayed.

Test Result

The main functions were tested using both valid and invalid inputs, and the expected outputs were checked.

12. Challenges Faced

- Designing the project by dividing it into separate Java classes while keeping the code simple.
- Correctly dealing with user input and invalid values.
- Putting into effect accurate calculations of attendance percentage and what-if scenarios.
- The identification of conflicts in the examination schedules.
- Add file handling to save and load data.
- The coordination of the various modules of the application.

13. Learnings & Key Takeaways

- I have understood the Java OOP principles and different objects.
- I have learned how ArrayList works and how to use dynamic data.
- I have developed skills in File Input and Output.

- I have learned how to use exceptions and validate input.
- I have learned how to divide the larger problem into smaller ones.
- I have enhanced my problem-solving capability by performing attendance analysis.
- I have gained some experience in testing.

14. Future Enhancements

- Create a web interface to simplify user interaction.
- Implement a database to manage data securely and scale it.
- Create a login page for students and faculty with role-based access.
- Include marks and result management features.
- Make attendance tracking possible on a subject-basis.
- Automate attendance and examination report generation.
- Create notifications for low attendance and examinations in advance.
- Use a calendar mode of exam scheduling.

15. References

1. VITyarthi
2. Oracle Corporation. **Java Documentation** – Java Programming Language, Collections, Exception Handling and File I/O.
3. Programming in Java – Course Materials.
4. Project Guidelines and Evaluation Criteria.